arXiv:2409.14660v3 [physics.flu-dyn] 25 Sep 2024

Fourier neural operators for spatiotemporal dynamics in two-dimensional turbulence

Mohammad Atif*, Pulkit Dubey†, Pratik P. Aghor‡, Vanessa López-Marrero*, Tao Zhang*, Abdullah Sharfuddin§, Kwangmin Yu*, Fan Yang*, Foluso Ladeinde§, Yangang Liu*, Meifeng Lin*, Lingda Li*

*Brookhaven National Laboratory, NY, USA
†University of New Hampshire, NH, USA
‡Georgia Institute of Technology, GA, USA
§Stony Brook University, NY, USA

***Abstract*—High-fidelity direct numerical simulation of turbulent flows for most real-world applications remains an outstanding computational challenge. Several machine learning approaches have recently been proposed to alleviate the computational cost even though they become unstable or unphysical for long time predictions. We identify that the Fourier neural operator (FNO) based models combined with a partial differential equation (PDE) solver can accelerate fluid dynamic simulations and thus address computational expense of large-scale turbulence simulations. We treat the FNO model on the same footing as a PDE solver and answer important questions about the volume and temporal resolution of data required to build pre-trained models for turbulence. We also discuss the pitfalls of purely data-driven approaches that need to be avoided by the machine learning models to become viable and competitive tools for long time simulations of turbulence.**

***Index Terms*—Machine Learning, Turbulence, Fourier Neural Operator**

I. Introduction

Machine learning (ML) has improved efficiency of forecasting in weather and climate models ([1], [2]). However, long time forecasts of ML-based emulators have been found to become unphysical or unstable (see [3] and references therein). Recently, it was shown in Ref. [4]) that the reason for this failure of ML-based emulators is spectral bias, where the smaller scales are not learned and only the large-scale dynamics are captured. In this paper, we propose a hybrid emulator for isotropic two-dimensional turbulence, which combines ML predictions with the governing partial differential equations (PDE) to march forward in time. This approach when generalized can be applied for long term predictions in climate models which are notoriously complex to solve due to the high computational cost of numerical PDE solvers. The key idea is to trade speed of the pure ML-emulator with the accuracy of the hybrid framework, i.e., the hybrid framework is not as fast as the pure ML-emulator but alternating between ML-emulator and PDE solver ensures that the results of the hybrid framework remain physical.

While the idea of using neural networks for flow field reconstruction has existed for over two decades [5], the application of machine learning in fluid dynamics witnessed a massive surge a decade ago [6]. Several works have since explored data-driven turbulence models, flow field reconstruction near walls, flow pattern recognition, machine learning accelerated simulations, and super-resolution (see [7]–[13] and references therein). Although, there exist several ML-based approaches for steady-state Reynolds-averaged Navier-Stokes simulations and the interpolation of flow fields in space and time, extrapolation problems have remained hard to tackle. This is particularly due to the nonlinear, multiscale, and chaotic nature of turbulence which remains a computational challenge for several important real-world applications of Navier-Stokes equations. The large memory requirement and scarcity of data have so far prohibited learning a generalized solution operator of the Navier-Stokes equations. Nevertheless, neural operators and embedding physical constraints in machine learning have shown promise in accelerating computational fluid dynamics.

In this paper, we evaluate the merits and discuss the pitfalls of a certain ML-based model for predicting spatiotemporal fluid dynamics. We choose to study decaying turbulence in two dimensions (2D) as it is an important problem and can be extended to forced turbulence or three dimensions. In Sec. II we briefly introduce the ML model. We adopt a purely data-driven approach and couple it with the numerical PDE solver to contain the growth of errors. It is unfair to expect a purely data-driven approach trained on a limited amount of data to accurately predict long term dynamics of a chaotic system. Thus, in Sec. III we discuss the data set and in Sec. IV assess the time over which a data-driven solver can be expected to make accurate predictions by calculating the Lyapunov exponents and computing evolution statistics of global quantities. The key question is how much data (samples and time refinement) is needed for an ML model to become a viable alternate to numerical solvers. Purely data-driven approaches are unlikely to replace the physical law-based numerical solvers [14], [15] as they lack the knowledge to model all relevant physics of a complex multiscale flow. Nevertheless, one needs to discuss its advantages and limitations by putting it on the same footing as numerical PDE solvers. To that end, in Sec. VI we identify an ML model, quantify the time refinement and number of samples (with different initial flow fields) required to train an ML model, and couple the ML model with PDE solver to make accurate long time predictions of the flow field

at the physical conditions and Reynolds number relevant to atmospheric physics [16].

II. Neural operators and related work

A neural operator is a neural network architecture that is designed to approximate a solution operator of resolution-independent PDEs. They learn mappings between infinite dimensional function spaces using a finite set of input-output data [17]–[21]. For example, the Fourier neural operator (FNO) operates in the frequency domain by learning mappings between Fourier coefficients of high-dimensional data [18], [20]. The deep network operator (DeepONet) [19] utilizes two subnetworks to encode solution operators to several deterministic and stochastic differential equations. The Laplace neural operator has been shown to work well with non-periodic functions [22]. The Markov neural operators are used to learn chaotic dynamics of non-ergodic dissipative systems [23]. The Clifford neural operators employ correlations between fields to learn multivector fields [24]. The Riemann operator network learns solution operators to hyperbolic PDEs with discontinuous solutions [25]. Additionally, there are several variations of each neural operator with their own special characteristics.

In this work, we select the Fourier neural operator (FNO) as it is a promising tool for learning discretization-agnostic approximations to the solution operator of differential equations. FNO truncates the high-frequency modes that are considered to be less important. Informally, if $\mathcal{A}$ and $\mathcal{U}$ are two Banach spaces of functions defined, respectively, on bounded domains $\Omega_{d_a} \subset \mathbb{R}^{d_a}$ and $\Omega_{d_u} \subset \mathbb{R}^{d_u}$, and $\mathcal{G}$ is an operator that maps functions $a \in \mathcal{A}$ to functions $u \in \mathcal{U}$, i.e., $\mathcal{G} : \mathcal{A} \to \mathcal{U}$, a FNO parameterizes the operator $\mathcal{G}$ by $\mathcal{G}_\theta : \mathcal{A} \times \theta \to \mathcal{U}$, such that for some $\theta^* \in \mathbb{R}^p$, we have the approximation $\mathcal{G}_{\theta^*} \approx \mathcal{G}$. Thus $\mathcal{G}_{\theta^*}$ acts as a surrogate to the operator $\mathcal{G}$. In the case of PDEs, the space $\mathcal{A}$ comprises "input" functions that the PDEs depend on, such as initial or boundary conditions, while the operator $\mathcal{G}$ maps these inputs to the solution $u \in \mathcal{U}$ satisfying the PDEs with the given inputs. In the present study, the input space $\mathcal{A}$ comprises initial conditions.

III. The Data Set

The data set consists of vorticity and velocity fields from 5000 simulations of 2D decaying turbulence. These simulations differ from one another in that they are initialized with different uniformly distributed random numbers but the same transport coefficient. The initial condition generates several opposite vortices that diffuse as time progresses and convect around the periodic domain depending on their initial locations. The Reynolds number is in the range of 7000-8000 for different samples depending on the initial condition. We solve the Navier-Stokes equations for the velocity field using the entropic lattice Boltzmann method [26], [27] on a grid of 256×256 points. The lattice Boltzmann method is an alternate methodology for computational fluid dynamics based on discrete space-time kinetic theory (see [28] for details). Each initial condition consumes 263 seconds on a single core of AMD EPYC 7402 CPU. The flow field is first allowed to evolve for $0.5\,t_c$ (where $t_c = L/U_0$ is the convection time and U_0 is the characteristic velocity) so that the initial sharp discontinuities vanish. Thereafter, time is reset to zero and the velocity ($\mathbf{u}$) and vorticity (ω_z) are sampled from time $t = 0$ to $t = t_c$ in steps of $0.005\,t_c$. Note that the vorticity field $\omega_z(x, y)$ is calculated as the curl of the velocity using $\omega_z(x, y) = \nabla \times \mathbf{u}(x, y)$, where ∇ is the nabla operator and the velocity vector $\mathbf{u}$ has components u_1, u_2 for two-dimensional fluid flow. Figure 8 visualizes the two dimensional vorticity field of one such sample where several inversely rotating vortices are visible. In general, in an incompressible viscous flow the vortices stretch (absent for two dimensions) and turn under the influence of each other's field and diffuse as the time advances.

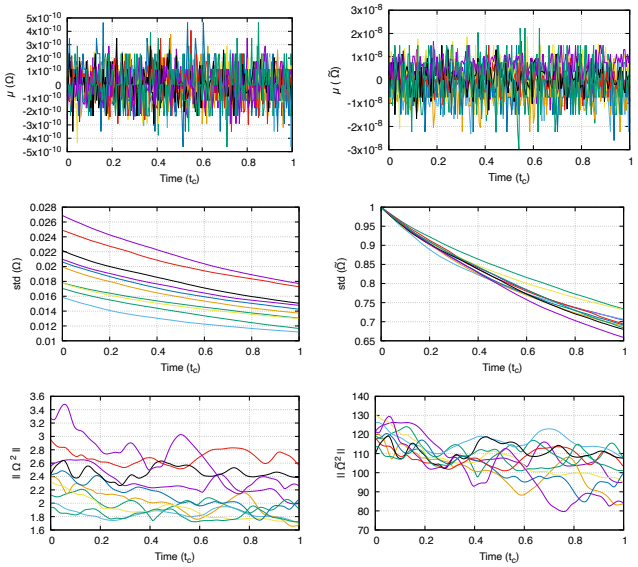

Fig. 1: Mean [top row], standard deviation [middle row], and Frobenius norm [bottom row] of raw vorticity (Ω) [left column] and normalized vorticity ($\tilde{\Omega}$) [right column]. The normalization is with respect to mean and standard deviation from $t = 0$. Each curve represents a different sample from the data set of 5000 samples.

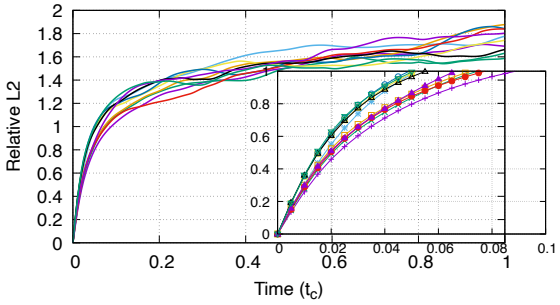

Fig. 2: L_2 norm of difference of vorticity fields of ten samples with their respective initial values.

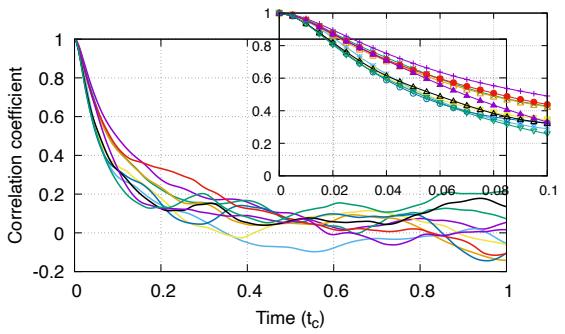

Fig. 3: Normalized projection of vorticity fields of the sample data sets on their respective initial values.

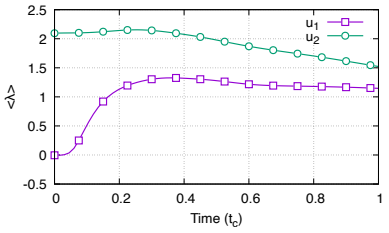

Fig. 4: Lyapunov exponents calculated for the two components of the velocity vector.

IV. Spatiotemporal Characteristics of the Flow Field

In general, turbulent dynamics are chaotic. Chaotic systems are characterized by sensitivity to initial conditions, i.e., two nearby initial conditions separate exponentially even when the underlying dynamical equations are deterministic ([29], [30]). For a given chaotic system, the Lyapunov time gives a time scale over which dynamics are predictable. Data-driven methods need to be mindful of this time scale to ensure accuracy and confidence when extrapolating in time. In this section, we first analyze the data set of 2D turbulence to understand the spatiotemporal characteristic of the flow field by computing the evolutions of the correlation coefficient and relative L_2 errors. The analysis ensures that there has been a meaningful evolution or separation from initial condition so that the prediction holds value, as, one pitfall is to make extremely short time predictions when the fields have evolved by such a tiny amount that even the initial condition would be an acceptable prediction. This is followed by estimating the Lyapunov time scale.

The evolution of volume mean of vorticity, standard deviation of vorticity, and global enstrophy (defined as the sum of square of vorticity fluctuation over the domain) with time for a subset of the full dataset has been shown in Fig. 1. The left column describes the behaviour of a data set as is, while the right column consists of the normalized datasets where the vorticity at each point in a sample has been scaled with the vorticity field's mean and standard deviation at the initial instant. The mean remains constant at zero due to incompressibility (thus vorticity fluctuation is the same as vorticity), while the standard deviation decays with time as is expected of decaying 2D turbulence. The global enstrophy computed from normalized vorticity also decays as small scale structures are dissipated due to viscosity. In order to emphasize the turbulent nature of the flow and determine time scales associated with chaotic dynamics, we plot in Figs. 2 and 3 two measures of time evolution for ten samples from the full dataset. Fig. 2 shows the separation between the vorticity fields for each dataset at time t from its initial value at time $t = 0$, scaled with their respective initial values. In Fig. 3, we plot the correlation coefficient of the vorticity field after time t and the initial vorticity field for the data sets scaled with their respective initial values. As expected, this correlation coefficient between the vorticity fields decays with increasing time.

For a dynamical system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$, in practice, we select two nearby initial conditions with initial separation denoted by $\delta x_0 = ||\delta \boldsymbol{x}(t = 0)||$, with a suitable norm chosen as measure of distance in the state space. As these trajectories evolve in time, we track their separation $\delta x(t) = ||\delta \boldsymbol{x}(t)||$ as a function of time. Sensitivity to initial conditions dictates that nearby initial conditions separate exponentially and can be expressed mathematically as $\delta x(t) \approx (\delta x_0) e^{\lambda t}$. In this work, we define Lyapunov time as $T_L = 1/\Lambda$, where Λ is the maximum Lyapunov exponent defined as $\Lambda = \max < \lambda >$, where

$$< \lambda >= \frac{\sum_i \lambda_i t_i}{\sum_i t_i}, \tag{1}$$

with $\lambda = (1/t) \ln (\delta x(t)/\delta x_0)$. To calculate Λ, we started with two initial conditions A and B such that $\delta x_0 = ||u_1^{(A)}(t = 0) - u_1^{(B)}(t = 0)|| = 10^{-2}$, with $||\cdot||$ being the L_2 norm. We then trace the evolution of $u_1^{(A)}, u_2^{(A)}, u_1^{(B)}, u_2^{(B)}$ and calculate the quantity λ_i at each time-step t_i for u_1 and u_2. Since the chaotic system has a finite maximum extent, the quantity λ_i grows as trajectories separate exponentially under chaotic dynamics until the separation between them becomes the size of the chaotic attractor itself in the state space. The larger of the two Λ's is ≈ 2.15 while their average is ≈ 1.7. Thus, a conservative estimate of the Lyapunov time is $T_L = \Lambda^{-1} \approx 0.45$ convective time–units. This is consistent with Fig. 3 where vorticity correlation coefficients are seen to decay until T_L after which the trajectories become independent. We will therefore restrict the predictions from FNO to a time horizon smaller than T_L.

V. Methods

In this section, we briefly introduce the methods for spatiotemporal dynamics of decaying turbulence. The goal is to find the best methodology for extrapolating the flow field in time from a few inherently chronological snapshots. The three methods that we consider in this paper are:

- **2D FNO with temporal channels:** Here, the two dimensions of FNO model spatial features. This is augmented

with channels (similar to the concept of RGB channels in a convolutional neural network for image data [31]) to include the time dimension where the channels are chronologically ordered. The number of input channels and output channels are parameters equal to number of input and output time snapshots respectively.

- **3D FNO:** In 3D FNO, two dimensions model the spatial features and one dimension models the temporal features. This does not differentiate between the spatial and temporal dimensions.
- **Hybrid FNO-PDE:** This method alternates between FNO and the PDE solver by using the output of FNO as input to PDE solver and vice versa. This could use either of the two FNO models listed above. It combines the speed of machine learning with the accurate physics of PDE solver. We leverage `torchScript` to execute this workflow [32].

In the above methods, we use PyTorch's neural operator library [33] for FNO and particle-resolved direct numerical simulation [16] as the `C++` PDE solver.

VI. Results

In this section, we study the errors of different FNO models for the spatiotemporal dynamics of decaying 2D turbulence. It should be noted here that the traditional PDE solvers have well understood time-stepping thresholds rooted in their stability analyses and requirements to capture the smallest scales of turbulence. However, in absence of an analogous thresholds for data-driven models one needs to rely on numerical experiments to decide the time resolution required for an accurate solution. Our previous study has demonstrated that a minimum of ten snapshots with a time resolution of $0.05\,t_c$ is necessary for making predictions at Reynolds number of 1000 [34]. In this paper, we increase the Reynolds number by approximately 10 times and refine the time resolution by a factor of 10. We will evaluate the models in terms of their number of parameters, training time, and error accumulated in long time roll-outs. In all the following studies, unless otherwise noted, the errors are reported as an average of 500 samples (all with different initial conditions) that were not a part of the training data.

A. 2D FNO with temporal channels

The 2D FNO model uses Fourier modes in the two spatial dimensions while stacking the time snapshots across channels with an inherent chronological ordering for temporal dimension. During training, the number of input channels is fixed to ten, but the number of output channels is varied from one to ten. The 2D FNO is used iteratively by using the outputs of the previous time as the input to ensure ten time snapshots are available as output when comparing errors. In order to ensure fairness, the models have been trained on equal volume of data. Note that when using fewer output channels, more training samples are generated using the same data volume.

We first plot the errors for different number of output channels in Fig. 5 for two widths 8 and 40. We notice that the errors are the largest for one output channel due to the "compound

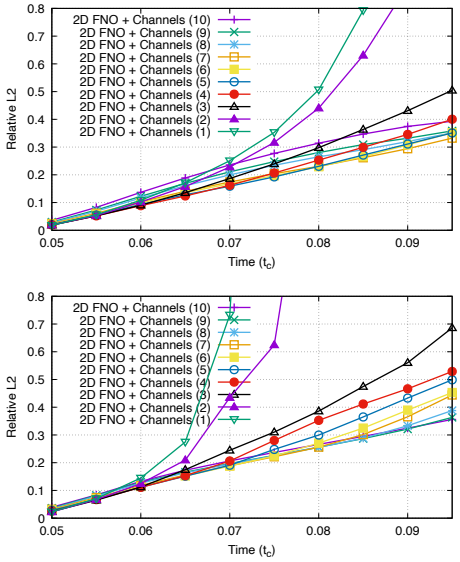

Fig. 5: Different number of channels with width 8 (top) and width 40 (bottom) while the other hyperparameters are: layers (4), modes (32), scheduler gamma (0.5), scheduler step (100), and learning rate (0.001).

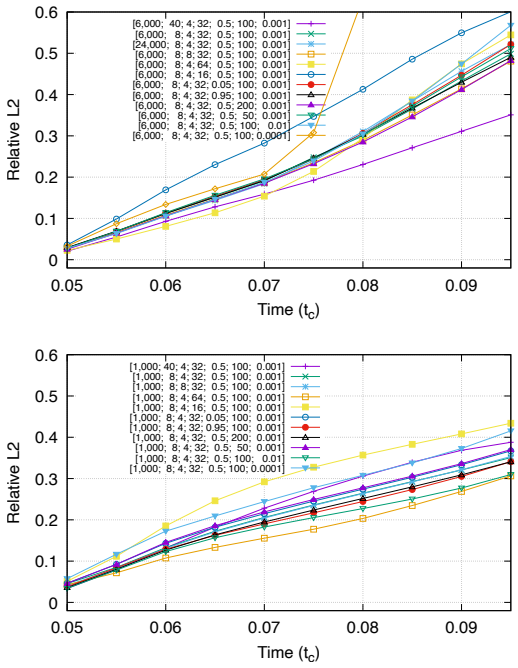

Fig. 6: Hyperparameter tuning for Channels (5) and (10): samples, width, layers, modes, scheduler gamma, scheduler step, learning rate.

Model	Width	Layers	Modes	Time (hours)	Training Size	Parameters
2D FNO + Channels (10)	40	4	32	2.41	1,000	6,995,922
2D FNO + Channels (10)	8	4	32	1.36	1,000	288,562
2D FNO + Channels (5)	40	4	32	7.25	6,000	6,994,637
2D FNO + Channels (5)	8	4	32	4.07	6,000	287,277
2D FNO + Channels (1)	40	4	32	11.48	10,000	6,993,609
2D FNO + Channels (1)	8	4	32	6.18	10,000	286,249
3D FNO	40	4	32	23.38	1,000	222,850,505
3D FNO	40	4	16	10.09	1,000	29,519,305
3D FNO	20	4	24	14.01	1,000	23,974,565
3D FNO	8	4	32	10.06	1,000	8,918,313
3D FNO	4	8	32	11.37	1,000	4,459,685
3D FNO	8	8	24	12.54	1,000	7,673,417

TABLE I: Total number of parameters in the model and their training time on Nvidia A6000.

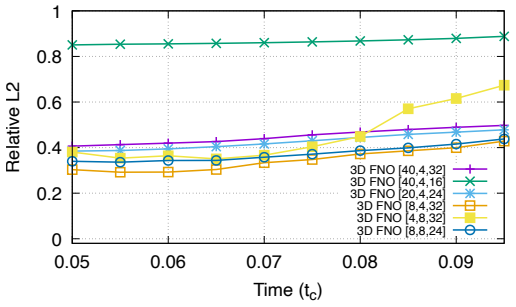

Fig. 7: Hyperparameter tuning of 3D FNO – the three most important hyperparameters are the width, number of layers, and number of Fourier modes in each dimension.

error" problem where prediction errors accumulate. The errors with width 40 for the same number of output channels are in general higher which suggests overfitting. The errors for channels 5 and 10 are compared in Fig. 6 for a wide range of hyperparameters from where it is seen that the errors are most sensitive to the number of Fourier modes.

B. 3D FNO

The 3D FNO models accept Fourier modes in all dimensions (2 spatial, 1 temporal) with 10 time snapshots each as input and output. Figure 7 compares the errors for several sets of hyperparameters. It is seen that the errors are most sensitive to the number of Fourier modes. It is also noticed that reducing the width improves the accuracy due to fewer parameters in the model (see Table I) which avoids the over-fitting problem. It is interesting to note from Figure 7 that the errors in 3D FNO models show weak dependence on time, in that they begin with large values and increase marginally as time progresses. From Table I it can also be noted that the training time of 3D FNO is larger than 2D FNO with channels. Therefore, 2D FNO with channels comes across as a better methodology for 2D turbulence due to its lower training time (hence the computational cost) and significantly lower errors for initial time steps.

C. Hybrid FNO-PDE for long-term temporal stability

As mentioned earlier, the aim of this paper is to assess the feasibility of FNO based models to augment numerical PDE solvers. Thus in this section we investigate the error accumulation of a hybrid FNO+PDE methodology and compare it with pure FNO and PDE solver. In the hybrid scheme, a single solver (numerical or ML-based ones) is invoked during a certain time window, and its output is used as the input of the other solver to make predictions for the next time window. These two solvers are used iteratively until the whole time period is solved. We choose the 2D FNO with 10 input channels (time $t = 0$ to $0.045t_c$) and 5 output channels ($t = 0.05t_c$ to $0.07t_c$) with the best set of hyperparameters for this comparison. The model is trained on velocity fields but we compute vorticity from the predicted velocity fields to ease visualization. This model thus acts as a pre-trained ML model for decaying 2D turbulence. Figure 8 visualizes vorticity fields from the three methodologies – PDE, 2D FNO with channels, and hybrid FNO-PDE. Figure 8 also plots the global values of kinetic energy, enstrophy, and divergence under different schemes for one sample. It is observed that the predictions from FNO are not divergence free (as the incompressibility of velocity fields was not incorporated in the loss function while training), but the PDE solver drives the fields toward divergence-free condition. Figure 9 plots the errors of kinetic energy and enstrophy for long term predictions and shows that errors from pure FNO get out of bound quickly while those from hybrid FNO + PDE remain stable. It is interesting to note that errors in kinetic energy remain smaller than 10% while the errors in enstrophy grow. This can be attributed to the fact that enstrophy is calculated from the gradient of velocity field while the model lacks any explicit mechanism to learn gradients. This could be addressed by incorporating governing equations in the loss functions or applying a relevant filter and will be investigated in future works.

VII. Discussion and Outlook

The numerical experiments in this paper lead to a viable hybrid methodology for accelerating direct numerical simulation by using FNO. We have demonstrated that the FNO when trained on a sufficiently large data set leads to a pre-trained model which when coupled with the numerical PDE solver can

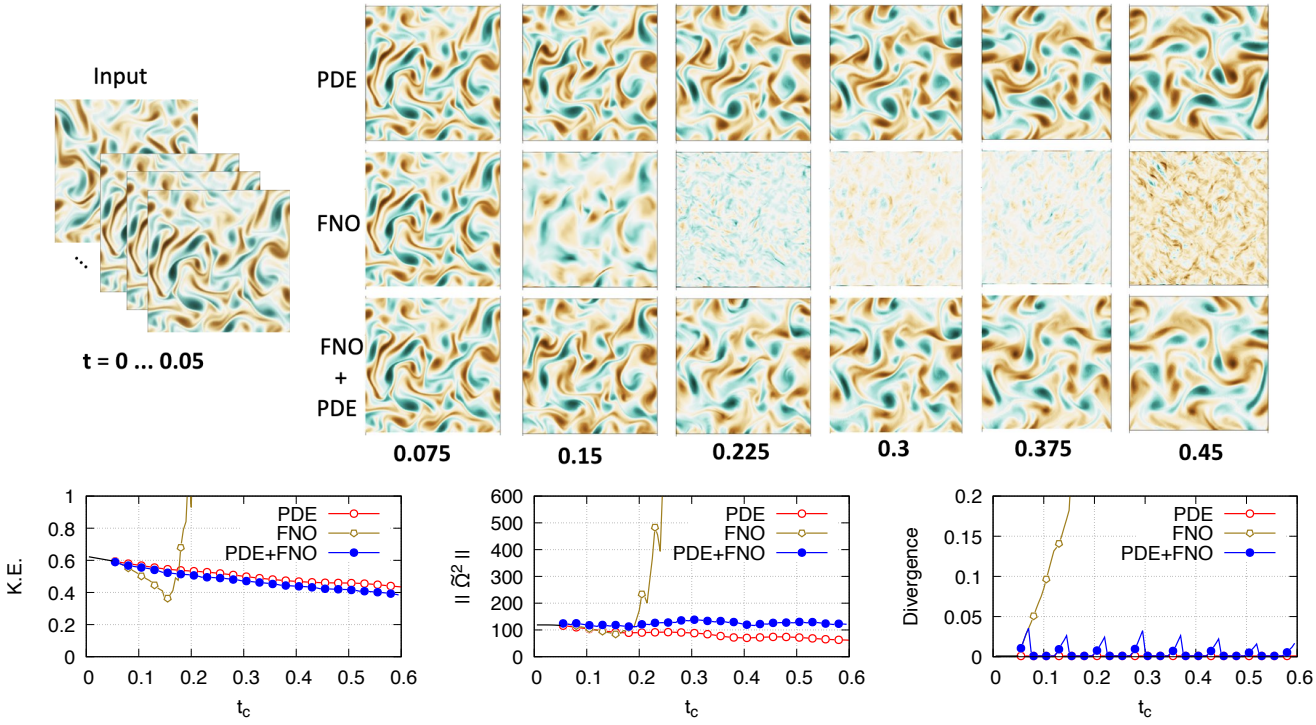

Fig. 8: Visualization of long time predictions of vorticity fields from three models [top], and global statistics of kinetic energy (K.E.), global enstrophy ($||\tilde{\Omega}^2||$), and divergence ($\nabla \cdot \boldsymbol{u}$) [bottom]. Times are in units of convection time t_c.

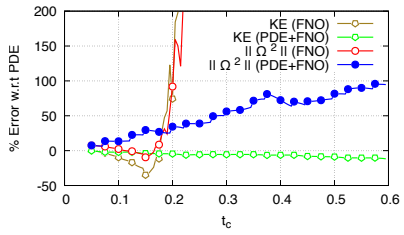

Fig. 9: Percentage errors in kinetic energy (K.E.) and enstrophy ($||\Omega^2||$) for long time predictions.

be used to make stable long-term predictions. The 3D FNO models were found to have a longer training time and yield larger errors for decaying 2D turbulence because the periodic spatial dimension cannot be treated at the same footing as non-periodic temporal dimensions. Thus, we conclude that stacking the temporal dimension across channels leads to smaller errors and requires a shorter training time.

It should also be noted that the FNO model generalizes well as it was trained on data set generated using lattice Boltzmann but coupled with finite difference based Navier-Stokes solver. This was possible as the physical conditions for both the numerical methods were identical by design, however, a word of caution is in order as the generalizability of pre-trained machine learning models has been discussed recently in the context of "foundational" models. The FNO models discussed in this work have been trained on the data of decaying 2D turbulence for a specific range of Reynolds number. In order to generalize it further as a solution operator for Navier-Stokes equation one needs to embed more physics in the training. Foundational models to be trained on sufficiently diverse data set and should at the minimum replicate canonical test cases of fluid dynamics. Their training also needs to incorporate the knowledge of governing equations as explored in many physics-informed machine learning works [35], [36]. An extension of the present framework to 3D should be straightforward with 3D FNO for spatial and channels for temporal dimensions.

High-fidelity direct numerical simulation of turbulent flows for most real-world applications remains an outstanding challenge due to its computational cost. Although, the advantage of machine learning inference manifests in the form of reduced time-to-solution, there exist several costs that are not frequently discussed such as – (i) the computational costs of training a model, (ii) procuring a GPU, and (iii) energy cost, some of which are amortized over several inference steps. In our hybrid PDE-FNO model presented here, the PDE solver takes 20 secs on AMD EPYC 7413 24-Core Processor for evolving flow field over $0.025\,t_c$. The machine learning component takes 0.1 secs for host to device data transfer and 0.3 secs for FNO inference on Nvidia A6000 GPU. However, the degree of optimization of the software libraries as well as scalability of the methodologies vary. Most high performance computing (HPC) codes are written in C++ while machine learning frameworks are Python based.

While coupling the two one also needs to account for the cost of transforming the data from C++ dynamic memory arrays into libTorch tensors. Several components discussed above often require optimization for real-world HPC applications which incur costs in terms of software redevelopment efforts. Thus, a fair comparison of the computational cost of different methodologies accounting for a wide range of metrics needs more research.

VIII. Acknowledgements

The authors gratefully acknowledge financial support from the Laboratory Directed Research and Development program at Brookhaven National Laboratory, which is sponsored by the US Department of Energy, Office of Science, under Contract Number DE-SC0012704. This research used resources of the National Energy Research Scientific Computing Center (NERSC), a Department of Energy Office of Science User Facility using NERSC award ASCR-ERCAP0027399.

References

[1] R. Lam, A. Sanchez-Gonzalez, M. Willson, P. Wirnsberger, M. Fortunato, F. Alet, S. Ravuri, T. Ewalds, Z. Eaton-Rosen, W. Hu *et al.*, "Learning skillful medium-range global weather forecasting," *Science*, vol. 382, no. 6677, pp. 1416–1421, 2023.

[2] J. Pathak, S. Subramanian, P. Harrington, S. Raja, A. Chattopadhyay, M. Mardani, T. Kurth, D. Hall, Z. Li, K. Azizzadenesheli *et al.*, "Fourcastnet: A global data-driven high-resolution weather model using adaptive fourier neural operators," *arXiv preprint arXiv:2202.11214*, 2022.

[3] C.-Y. Lai, P. Hassanzadeh, A. Sheshadri, M. Sonnewald, R. Ferrari, and V. Balaji, "Machine learning for climate physics and simulations," *arXiv preprint arXiv:2404.13227*, 2024.

[4] A. Chattopadhyay and P. Hassanzadeh, "Long-term instabilities of deep learning-based digital twins of the climate system: The cause and a solution," *arXiv preprint arXiv:2304.07029*, 2023.

[5] M. Milano and P. Koumoutsakos, "Neural network modeling for near wall turbulent flow," *Journal of Computational Physics*, vol. 182, no. 1, pp. 1–26, 2002.

[6] J. Ling, A. Kurzawski, and J. Templeton, "Reynolds averaged turbulence modelling using deep neural networks with embedded invariance," *Journal of Fluid Mechanics*, vol. 807, p. 155–166, 2016.

[7] S. Bhushan, G. W. Burgreen, D. Martinez, and W. Brewer, "Machine learning for turbulence modeling and predictions," in *Fluids Engineering Division Summer Meeting*, vol. 83730. American Society of Mechanical Engineers, 2020, p. V003T05A008.

[8] K. Fukami, K. Fukagata, and K. Taira, "Machine-learning-based spatio-temporal super resolution reconstruction of turbulent flows," *Journal of Fluid Mechanics*, vol. 909, p. A9, 2021.

[9] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer, "Machine learning–accelerated computational fluid dynamics," *Proceedings of the National Academy of Sciences*, vol. 118, no. 21, p. e2101784118, 2021.

[10] R. Vinuesa and S. L. Brunton, "The potential of machine learning to enhance computational fluid dynamics," *arXiv preprint arXiv:2110.02085*, pp. 1–13, 2021.

[11] O. Obiols-Sales, A. Vishnu, N. Malaya, and A. Chandramowliswharan, "CFDNet: A deep learning-based accelerator for fluid simulations," in *Proceedings of the 34th ACM international conference on supercomputing*, 2020, pp. 1–12.

[12] T. Wang, B. Wang, E. S. Matekole, and M. Atif, "Finding hidden patterns in high resolution wind flow model simulations," in *Smoky Mountains Computational Sciences and Engineering Conference*. Springer, 2022, pp. 345–365.

[13] M. Lino, S. Fotiadis, A. A. Bharath, and C. D. Cantwell, "Current and emerging deep-learning methods for the simulation of fluid dynamics," *Proceedings of the Royal Society A*, vol. 479, no. 2275, p. 20230058, 2023.

[14] S. Succi and P. V. Coveney, "Big data: the end of the scientific method?" *Philosophical Transactions of the Royal Society A*, vol. 377, no. 2142, p. 20180145, 2019.

[15] H. Viswanath, M. A. Rahman, A. Vyas, A. Shor, B. Medeiros, S. Hernandez, S. E. Prameela, and A. Bera, "Neural operator: Is data all you need to model the world? an insight into the impact of physics informed machine learning," *arXiv preprint arXiv:2301.13331*, 2023.

[16] Z. Gao, Y. Liu, X. Li, and C. Lu, "Investigation of turbulent entrainment-mixing processes with a new particle-resolved direct numerical simulation model," *J. Geophys. Res. Atmos.*, vol. 123, no. 4, pp. 2194–2214, 2018.

[17] T. Chen and H. Chen, "Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems," *IEEE Transactions on Neural Networks*, vol. 6, no. 4, pp. 911–917, 1995.

[18] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," *arXiv:2010.08895*, 2020.

[19] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators," *Nat Mach Intell*, vol. 3, pp. 218–229, 2021, https://doi.org/10.1038/s42256-021-00302-5.

[20] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Neural operator: Learning maps between function spaces with applications to PDEs," *Journal of Machine Learning Research*, vol. 24, no. 89, pp. 1–97, 2023, https://www.jmlr.org/papers/volume24/21-1524/21-1524.pdf.

[21] T. Zhang, L. Li, V. López-Marrero, M. Lin, Y. Liu, F. Yang, K. Yu, and M. Atif, "Emulator of pr-dns: Accelerating dynamical fields with neural operators in particle-resolved direct numerical simulation," *Journal of Advances in Modeling Earth Systems*, vol. 16, no. 2, p. e2023MS003898, 2024.

[22] Q. Cao, S. Goswami, and G. E. Karniadakis, "Laplace neural operator for solving differential equations," *Nature Machine Intelligence*, pp. 1–10, 2024.

[23] Z. Li, M. Liu-Schiaffini, N. Kovachki, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, "Learning dissipative dynamics in chaotic systems," *arXiv preprint arXiv:2106.06898*, 2021.

[24] J. Brandstetter, R. v. d. Berg, M. Welling, and J. K. Gupta, "Clifford neural layers for pde modeling," *arXiv preprint arXiv:2209.04934*, 2022.

[25] A. Peyvan, V. Oommen, A. D. Jagtap, and G. E. Karniadakis, "Riemannonets: Interpretable neural operators for Riemann problems," *Computer Methods in Applied Mechanics and Engineering*, vol. 426, p. 116996, 2024.

[26] M. Atif, P. K. Kolluru, C. Thantanapally, and S. Ansumali, "Essentially entropic lattice boltzmann model," *Phys. Rev. Lett.*, vol. 119, p. 240602, Dec 2017. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.119.240602

[27] M. Atif, N. Maruthi, P. Kolluru, C. Thantanapally, and S. Ansumali, "Lattice Boltzmann model for transonic flows," *arXiv preprint arXiv:2409.05114*, 2024.

[28] M. Atif, P. K. Kolluru, and S. Ansumali, "Essentially entropic lattice boltzmann model: Theory and simulations," *Phys. Rev. E*, vol. 106, p. 055307, Nov 2022. [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevE.106.055307

[29] S. H. Strogatz, *Nonlinear dynamics and chaos: with applications to physics, biology, chemistry, and engineering*. CRC press, 2018.

[30] P. Cvitanovic, R. Artuso, R. Mainieri, G. Tanner, G. Vattay, N. Whelan, and A. Wirzba, "Chaos: classical and quantum," *ChaosBook. org (Niels Bohr Institute, Copenhagen 2005)*, vol. 69, p. 25, 2005.

[31] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, 2012.

[32] Z. DeVito, "Torchscript: Optimized execution of pytorch programs," *Retrieved January*, 2022.

[33] "Neuraloperator: Learning neural operators in pytorch," https://github.com/neuraloperator/neuraloperator.

[34] M. Atif, V. López-Marrero, T. Zhang, A. A. M. Sharfuddin, K. Yu, J. Yang, F. Yang, F. Ladeinde, Y. Liu, M. Lin *et al.*, "Towards accelerating particle-resolved direct numerical simulation with neural operators," *Statistical Analysis and Data Mining: The ASA Data Science Journal*, vol. 17, no. 3, p. e11690, 2024.

[35] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, 2021.

[36] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, and A. Anandkumar, "Physics-informed neural operator for learning partial differential equations," *ACM/JMS Journal of Data Science*, vol. 1, no. 3, pp. 1–27, 2024.